AN12731

8-bit CRC calculation procedure using SPI communication for the FXPS7xxxD4 pressure sensor

Rev. 1 — 1 April 2020 Application note

1 Introduction

The FXPS7xxxD4 device family includes a standard SPI protocol requiring 32-bit data packets. The FXPS7xxxD4 is a slave device requiring the base clock value be low (CPOL = 0) with data captured on the rising edge of the clock and data propagated on the falling edge of the clock (CPHA = 0). The most significant bit is transferred first (MSB first). As shown in Figure 1, SPI transfers are completed through a sequence of two phases. During the first phase, the command is transmitted from the SPI master to the slave device. During the second phase, response data is transmitted from the slave device to the SPI master. MOSI and SCLK transitions are ignored when SS_B is not asserted.

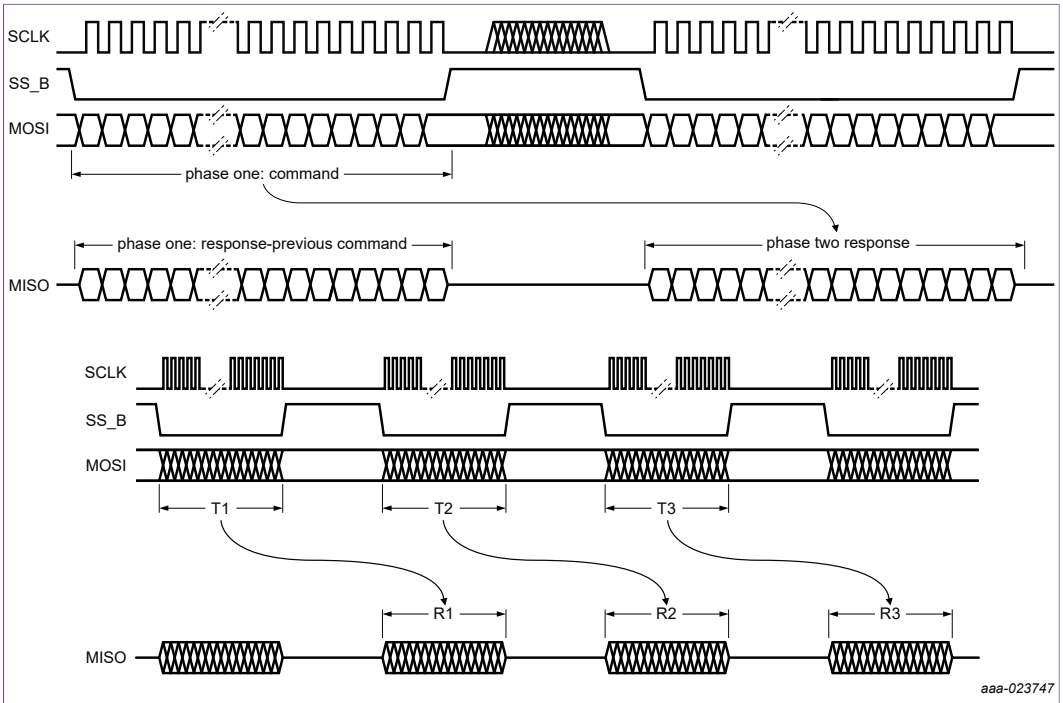

Figure 1. Standard 32-bit SPI protocol timing diagram

The following sections provide the procedures and software examples to compute a proper 8-bit CRC, insert the calculated CRC to the message, and send the message over the SPI. As a reference, verification of the 8-bit CRC calculation can be checked using the four examples shown in Table 2. Section 3 and Section 4 describe the read and write register commands and their respective responses, as well as show the SPI bus traffic during the process. Finally, Section 5 describes the sensor data request command and response examples along with their SPI bus traffic during the process.

2 Procedure for 8-bit CRC

The device calculates an 8-bit CRC on the entire 32-bits of each command. Message data is entered into the CRC calculator MSB first, consistent with the transmission order of the message. If the calculated CRC does not match the transmitted CRC, the command is ignored and the device responds with the SPI Error. Table 1 shows the CRC polynomial and seed.

Table 1. CRC polynomial and seed

SPICRCSEED[3:0]	Default Polynomial	Default Non-Direct Seed
0000	$X^8 + X^5 + X^3 + X^2 + X + 1$	1111 1111

The code snippet, crc.h, provides the CRC calculation flow the user needs to add into their project in order to calculate the 8-bit CRC while creating the register read/write or sensor data request command messages. The response message received from the slave includes an 8-CRC which must be checked. This response CRC is checked in the bCrc_Check function call also included in the coded snippet crc.h.

Note: *The code snippets provided in this document are BSD-3 clause licensed. Refer to* https://opensource.org/licenses/BSD-3-Clause *for more information.*

```
#include crc.h

/* Declare CRC parameter instance and initialize with
   following values
*/
tCrcParameters crc_calc_param =
{
  .eType          = eSeedType_NonDirect,
  .u16Polynomial  = 0x012F, /* Default polynomial*/
  .u8InitialValue = 0xFF,
  .u8CrcLength    = 0x08,
  .u8DataBitStart = 0,
  .u8DataLength   = 32
};

/* Declare 32-bit command message and response variables */
uint32_t u32_cmd_msg = 0x00000000;
uint32_t u32_response;

/* Declare variable to check CRC of the response message */
uint8_t responseCrc;

/* Create register read/write or sensor data request command
   message. Refer to the "FXPS7xxxD4" data sheet, Standard
   32-bit SPI protocol section and prepare following:
   "Command" (Register read or write or sensor data request
    command)

   "RegisterAddress" (Register address to read from or write
    to, in case of sensor data request it will be 0x00)

   "RegisterData" (Value write to register address, in case of
    register read and sensor data request it will be 0x00)
*/

/* Insert "Command" value into the command message */
```


All information provided in this document is subject to legal disclaimers.

© NXP B.V. 2020. All rights reserved.

```
u32_cmd_msg |= ((Command << 28) & 0xF0000000);

/* Insert "Register Address" value into the command message */
u32_cmd_msg |= ((RegisterAddress << 16) & 0x00FF0000);

/* Insert "Register Data" value into the command message */
u32_cmd_msg |= ((RegisterData << 8) & 0xF0000000);

/* Keep the CRC field in the command message as 0x00, since
   u32_cmd_msg was initialized to zero it took care of keeping
   CRC field as 0x00

   Call CRC Calculate Function available in crc.c
   Store returned 8-bit CRC calculated valued and insert into
   the command message
*/
u32_cmd_msg |= ((u8Crc_Calculate(crc_calc_param, u32_cmd_msg))
 & 0x000000FF);

/* Transfer generated command message i.e. "u32_cmd_msg" using
   32-bit SPI transfer function
   Note: Use 32-bit SPI bidirectional data transfer API for
   the MCU in use. Pass "u32_response" to store the 32-bit
   response message received back from device. Refer
   to underlying MCU SDK APIs for SPI peripheral.
*/

/* Check CRC value of the received response message */
/* Call CRC Check function available in crc.c */
responseCrc = bCrc_Check((u32_response & 0xFF),
 crc_calc_param, (u32_response & 0xFFFFFF00));
```

The CRC calculation is found in the following code snippet. Users call these functions in the CRC calculation flow described above.

```
//////////////////////////////////////////////////////////////////
// Copyright (c) 2018, 2020, NXP Semiconductors.
// All rights reserved.
//
// Redistribution and use in source and binary forms, with or
// without modification, are permitted provided that the
// following conditions are met:
//     * Redistributions of source code must retain the above
//       copyright notice, this list of conditions and the
//       following disclaimer.
//     * Redistributions in binary form must reproduce the
//       above copyright notice, this list of conditions and
//       the following disclaimer in the documentation
//       and/or other materials provided with the distribution.
//     * Neither the name of NXP Semiconductors nor the names
//       of its contributors may be used to endorse or promote
//       products derived from this software without specific
//       prior written permission.
//
// THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND
// CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES,
// INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF
// MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE
// DISCLAIMED. IN NO EVENT SHALL NXP SEMICONDUCTORS BE LIABLE
// FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY,
```


All information provided in this document is subject to legal disclaimers.

© NXP B.V. 2020. All rights reserved.

```
// OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO,
// PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
// DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED
// AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,
// STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR
// OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS
// SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

#ifndef CRC_H_INCLUDED
#define CRC_H_INCLUDED

/////////////////////////////////////////////////////////////////
//!                          Includes
/////////////////////////////////////////////////////////////////
#include <stdint.h>

/////////////////////////////////////////////////////////////////
//!                        Enumeration
/////////////////////////////////////////////////////////////////
typedef enum
{
  eSeedType_Direct, // not supported yet
  eSeedType_NonDirect
} eSeedType;

/////////////////////////////////////////////////////////////////
//!                        Structure
/////////////////////////////////////////////////////////////////
typedef struct
{
  eSeedType  eType;
  uint8_t    u8InitialValue;
  uint16_t   u16Polynomial;
  uint8_t    u8CrcLength;
  uint8_t    u8DataBitStart;
  uint8_t    u8DataLength; // including the CRC
} tCrcParameters;
/////////////////////////////////////////////////////////////////
//!                  Public prototype
/////////////////////////////////////////////////////////////////
uint8_t u8Crc_Calculate(tCrcParameters crc_calc_param,
 uint32_t u32_msg);
bool bCrc_Check(uint8_t u8_crc, tCrcParameters crc_calc_param,
 uint32_t u32_response);

#endif //CRC_H_INCLUDED
```

The CRC check function body is found in the following code snippet. Users call these functions in the previously described CRC calculation flow.

```
/////////////////////////////////////////////////////////////////
// Copyright (c) 2018,2020, NXP Semiconductors.
// All rights reserved.
//
// Redistribution and use in source and binary forms, with or
// without modification, are permitted provided that the
// following conditions are met:
//     * Redistributions of source code must retain the
//       above copyright notice, this list of conditions
//       and the following disclaimer.
```


All information provided in this document is subject to legal disclaimers.

© NXP B.V. 2020. All rights reserved.

```
//      * Redistributions in binary form must reproduce the
//        above copyright notice, this list of conditions
//        and the following disclaimer in the documentation
//        and/or other materials provided with the distribution.
//      * Neither the name of NXP Semiconductors nor the names
//        of its contributors may be used to endorse or promote
//        products derived from this software without specific
//        prior written permission.
//
// THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND
// CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES,
// INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF
// MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE
// DISCLAIMED. IN NO EVENT SHALL NXP SEMICONDUCTORS BE LIABLE
// FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY,
// OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO,
// PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
// DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER
// CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN
// CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE
// OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS
// SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

////////////////////////////////////////////////////////////////
//!                          Includes
////////////////////////////////////////////////////////////////
#include "Crc.h"

////////////////////////////////////////////////////////////////
//!                          Methods
////////////////////////////////////////////////////////////////
uint8_t u8Crc_Calculate(tCrcParameters crc_calc_param, uint32_t
 u32_cmd_msg)
{
   uint16_t u16_index;

/* Calculate mask value */
uint64_t u64_mask = ((~(uint64_t)0) >> (32 -
 crc_calc_param.u8DataLength)) - ((uint64_t)(((uint64_t)1) <<
 crc_calc_param.u8CrcLength) - 1);

/* Update the command message using CRC params */
u32_cmd_msg = (u32_cmd_msg >> crc_calc_param.u8DataBitStart) &
 ((uint32_t)(((uint64_t)1) << crc_calc_param.u8DataLength)
 - 1);

/* Calculate remainder and XOR value */
uint64_t u64_remainder = (uint64_t)(((((uint64_t)u32_cmd_msg))
 | (((uint64_t)crc_calc_param.u8InitialValue) <<
 (crc_calc_param.u8DataLength))) & u64_mask);
 uint64_t u64_xor = ((uint64_t)crc_calc_param.u16Polynomial)
 <<
 (crc_calc_param.u8DataLength - 1) ;

/* Calculate remainder by shifting xor value over message
   and CRC length */
for (u16_index = (uint16_t)
 (((uint16_t)crc_calc_param.u8DataLength) +
 ((uint16_t)crc_calc_param.u8CrcLength)); u16_index > 0;
 u16_index--)
```


All information provided in this document is subject to legal disclaimers.

© NXP B.V. 2020. All rights reserved.

```
{
 if (((u64_remainder >> (u16_index - 1)) & 0x01) != 0)
 {
    u64_remainder ^= u64_xor;
 }
 u64_xor >>= 1;
 if ((u64_remainder & u64_mask) == 0)
 {
     break;
 }
}

/* Return 8-bit CRC calculated */
return (uint8_t)(u64_remainder & (uint8_t)((uint64_t)
 (((uint64_t)1) << crc_calc_param.u8CrcLength) - 1));
}

bool bCrc_Check(uint8_t u8_crc, tCrcParameters crc_calc_param,
 uint32_t u32_response)
{
/* Check whether calculated 8-bit CRC on response message
   matches with u8_crc */
return u8_crc == u8Crc_Calculate(crc_calc_param, u32_response);
}
```

Example CRC calculations are shown in Table 2.

Table 2. Command and response mode - CRC calculation examples

Physical address	Command	Extended data	Regular data	Non-direct Seed	8-bit CRC
0x01	0x08	0x11	0x86	0xFF	0xB0
0x02	0x01	0x25	0xFF	0xFF	0x38
0x03	0x0F	0x1A	0x41	0xFF	0x2C
0x04	0x01	0x01	0x01	0xFF	0xD4

3 Register read command

The device supports a register read command. The register read command uses the upper 7 bits of the addresses to address two 8-bit registers in the register map. The response to the command includes the contents of RA[7:1] high byte (RA[0] = 1) in the upper byte and the contents of RA[7:1] low byte (RA[0] = 0) in the lower byte. The register write is executed and a response is transmitted on the next SPI message.

3.1 Register read command message format

Table 3. Register read command message format

31 (MSB)	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Command C[3:0]				**0**	**0**	**0**	**0**	**Register address**							
1	1	0	0	0	0	0	0	RA[7:1]							RA[0]

All information provided in this document is subject to legal disclaimers.

© NXP B.V. 2020. All rights reserved.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0 (LSB)
Register data								**8-bit CRC**							
0	0	0	0	0	0	0	0	CRC[7:0]							

Bit field	Definition
C[3:0]	Register read command = '1100'.
RA[7:1]	RA[7:1] contains the word address of the register to be read.
CRC[7:0]	CRC: See Section 2 for code snippet calculation.

Note: *Whether reading or writing, the response will always return two bytes; odd byte and even byte. For example, if reading 0x18, the response gives the contents of both 0x17 and 0x18. If reading 0x17, the response gives the contents of of both 0x17 and 0x18.*

3.2 Register read response message format

Table 4. Register read response message format

31 (MSB)	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Command C[0], [3:1]				**Basic status**		**0**	**0**	**Register data: Contents of RA[7:1] High byte**							
0	1	1	0	ST[1:0]		0	0	RD[15:8]							

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0 (LSB)
Register data: Contents of RA[7:1] Low byte								**8-bit CRC**							
RD[7:0]								CRC[7:0]							

Bit field	Definition
C[0], [3,1]	Register read command - '0110'
ST[1:0]	Status. Refer to Table 30, *Basic status field for responses to register commands* and Table 31, *Error responses bit field descriptions* in the FXPS7xxxD4 data sheets.
RD[15:8]	The contents of the register addressed by RA[7:1] High byte (RA[0] = 1)
RD[7:0]	The contents of the register addressed by RA[7:1] Low byte (RA[0] = 0)
CRC[7:0]	CRC: See Section 2 for code snippet calculation.

Figure 2 illustrates a register read command of the registers 0x3E and 0x3F (WHO_AM_I and I2C_ADDRESS):

© NXP B.V. 2020. All rights reserved.

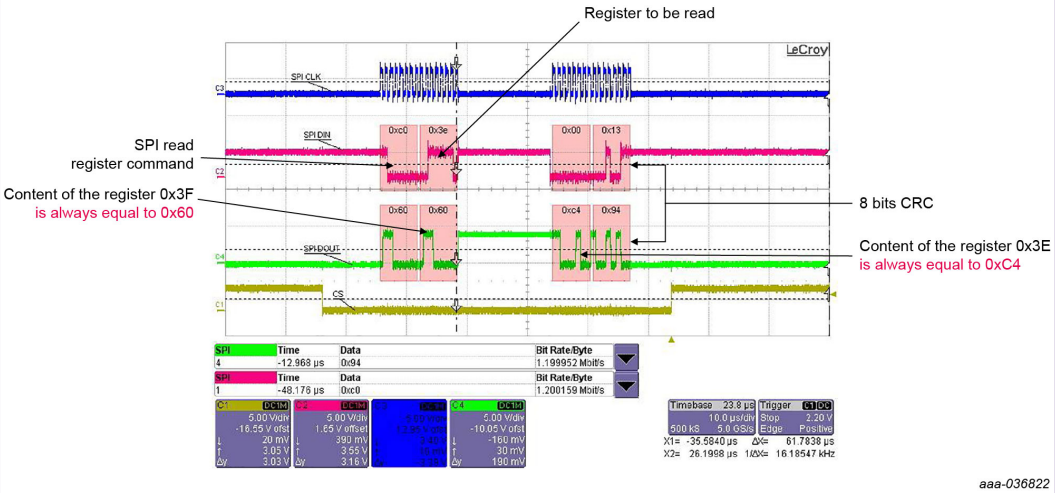

Figure 2. Register read command of 0x3E and 0x3F (WHO_AM_I and I2C_ADDRESS)

4 Register write command

The device supports a register write command. The register write command writes the value specified in RD[7:0] to the register addressed by RA[7:0]. The response to the command includes the new contents of RA [7:1] high byte (RA[0] = 1) in the upper byte and the contents of RA[7:1] low byte (RA[0] = 0) in the lower byte. The register write is executed and a response is transmitted on the next SPI message.

4.1 Register write command message format

Table 5. Register write command message format

31 (MSB)	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Command C[3:0]				**0**	**0**	**0**	**0**	**Register address**							
1	0	0	0	0	0	0	0	RA[7:1]							RA[0]

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0 (LSB)
Register data								**8-bit CRC**							
RD[7:0]								CRC[7:0]							

Bit field	Definition
C[3:0]	Register read command = '1000'.
RA[7:0]	RA[7:0] contains the address of the register to be written.
RD[7:0]	RD[7:0] contains the data byte to be written to address RA[7:0].
CRC[7:0]	CRC: See Section 2 for code snippet calculation.

All information provided in this document is subject to legal disclaimers.

© NXP B.V. 2020. All rights reserved.

4.2 Register write response message format

Table 6. Register write response message format

31 (MSB)	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Command C[0], [3:1]				**Basic status**		**0**	**0**	**Register data: Contents of RA[7:1] High byte**							
0	1	0	0	ST[1:0]		0	0	RD[15:8]							

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0 (LSB)
Register data: Contents of RA[7:1] Low byte								**8-bit CRC**							
RD[7:0]								CRC[7:0]							

Bit field	Definition
C[0], [3,1]	Register read command - '0100'.
ST[1:0]	Status. Refer to Table 30, *Basic status field for responses to register commands* and Table 31, *Error responses bit field descriptions* in the FXPS7xxxD4 data sheets.
RD[15:8]	The contents of the register addressed by RA[7:1] High byte (RA[0] = 1).
RD[7:0]	The contents of the register addressed by RA[7:1] Low byte (RA[0] = 0).
CRC[7:0]	CRC: See Section 2 for code snippet calculation.

Figure 3 illustrates a register write command of 0x24 in the register 0x42:

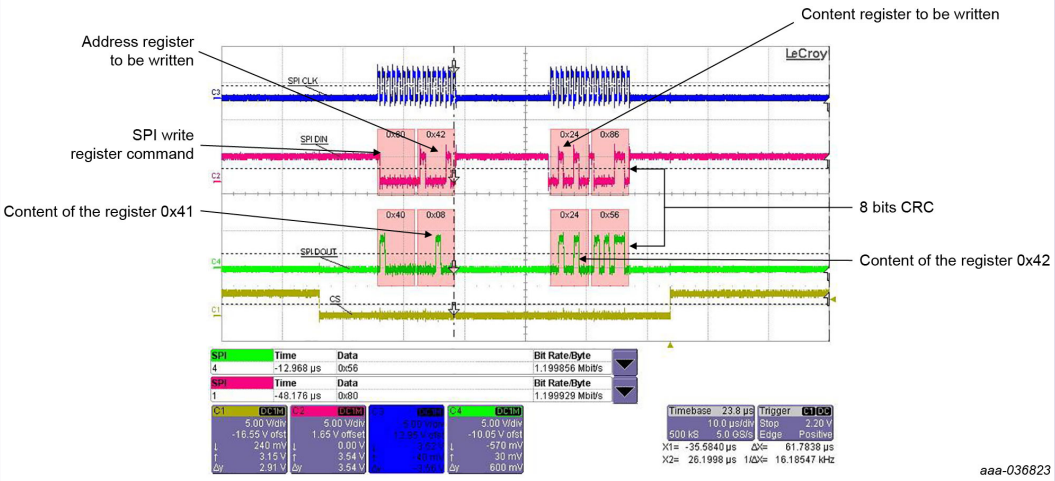

Figure 3. Register write command of 0x24 in the register 0x42

5 Data request command (direct access to sensor data pressure)

The device supports standard sensor data request commands. The sensor data request command format is described in Section 5.1. The response to a sensor data request is shown in Section 5.2. The response is transmitted on the next SPI message subject to the error handling conditions. The sensor data included in the response is the sensor data at the falling edge of SS_B for the sensor data request response.

All information provided in this document is subject to legal disclaimers.

© NXP B.V. 2020. All rights reserved.

5.1 Sensor data request command message format

Table 7. Sensor data request command message format

31 (MSB)	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Command				**0**	**0**	**0**	**0**	0	0	0	0	0	0	0	0
C[3:0]				0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0 (LSB)
0	0	0	0	0	0	0	0	**8-bit CRC**							
0	0	0	0	0	0	0	0	CRC[7:0]							

Bit field	Definition
C[0]	Sensor data request commend = '1'.
C[3:1] = SOURCEID[2:0]	Source identification code for the requested sensor data. See Source Identification registers, field SOURCEID_x and the SPI command bit allocation table in the FXPS7xxxD4 data sheet.
CRC[7:0]	CRC: See Section 2 for code snippet calculation..

Note: The data request command depends on the sensor source ID which can be configured on register 0x1A and 0x1B.

5.2 Sensor data request response message format

Table 8. Sensor data request response message format

31 (MSB)	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Command C[0], [3:1]				**Basic status**		**Sensor data**									
1	C[3]	C[2]	C[1]	ST[1:0]		SD[11:0]									

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0 (LSB)
Sensor data						**Detail status**		**8-bit CRC**							
SD[11:0]		0	0	0	0	SF[1:0]		CRC[7:0]							

All information provided in this document is subject to legal disclaimers.

© NXP B.V. 2020. All rights reserved.

Bit field	Definition
C[0]	Sensor data request command = '1'.
C[3:1] = SOURCE[2:0]	Source identification code for the requested sensor data. Refer to Table 30, *Basic status field for responses to register commands* and Table 31, *Error responses bit field descriptions* in the FXPS7xxxD4 data sheets.
ST[1:0]	Basic status. Refer to Table 30, *Basic status field for responses to register commands* and Table 31, *Error responses bit field descriptions* in the FXPS7xxxD4 data sheets.
SD[11:0]	Sensor data. Refer to the absolute pressure output data scaling equation section in the FXPS7xxxD4 data sheets.
SF[1:0]	Detailed status. Refer to Table 30, *Basic status field for responses to register commands* and Table 31, *Error responses bit field descriptions* in the FXPS7xxxD4 data sheets.
CRC[7:0]	CRC: See Section 2 for code snippet calculation.

Figure 4 illustrates a data request command with a sensor source ID = 1:

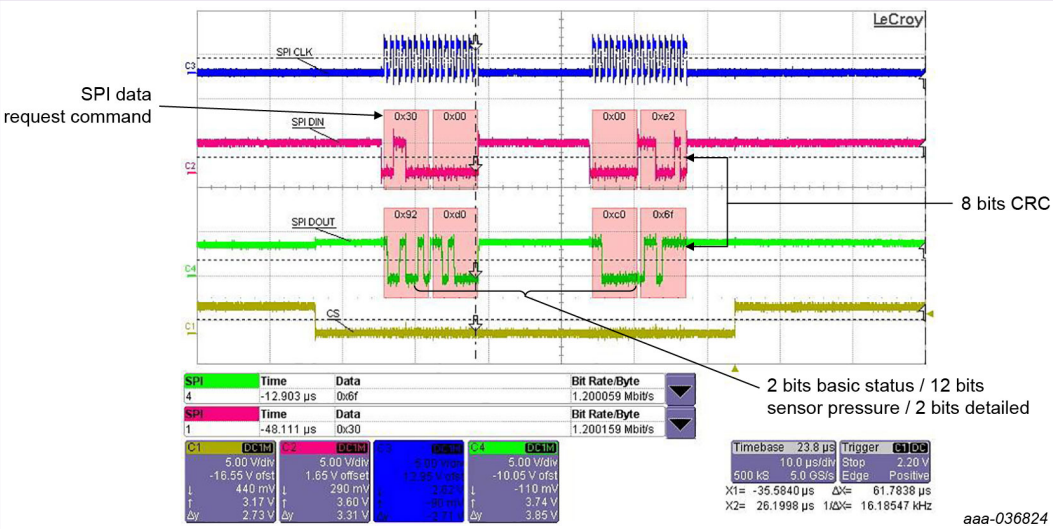

Figure 4. Data request command with a sensor source ID = 1

6 Revision history

Table 9. Revision history

Revision	Date	Description
AN12731 v.1	20200401	Initial release

All information provided in this document is subject to legal disclaimers.

© NXP B.V. 2020. All rights reserved.

7 Legal information

7.1 Definitions

Draft — The document is a draft version only. The content is still under internal review and subject to formal approval, which may result in modifications or additions. NXP Semiconductors does not give any representations or warranties as to the accuracy or completeness of information included herein and shall have no liability for the consequences of use of such information.

7.2 Disclaimers

Limited warranty and liability — Information in this document is believed to be accurate and reliable. However, NXP Semiconductors does not give any representations or warranties, expressed or implied, as to the accuracy or completeness of such information and shall have no liability for the consequences of use of such information. NXP Semiconductors takes no responsibility for the content in this document if provided by an information source outside of NXP Semiconductors. In no event shall NXP Semiconductors be liable for any indirect, incidental, punitive, special or consequential damages (including - without limitation - lost profits, lost savings, business interruption, costs related to the removal or replacement of any products or rework charges) whether or not such damages are based on tort (including negligence), warranty, breach of contract or any other legal theory. Notwithstanding any damages that customer might incur for any reason whatsoever, NXP Semiconductors' aggregate and cumulative liability towards customer for the products described herein shall be limited in accordance with the Terms and conditions of commercial sale of NXP Semiconductors.

Right to make changes — NXP Semiconductors reserves the right to make changes to information published in this document, including without limitation specifications and product descriptions, at any time and without notice. This document supersedes and replaces all information supplied prior to the publication hereof.

Applications — Applications that are described herein for any of these products are for illustrative purposes only. NXP Semiconductors makes no representation or warranty that such applications will be suitable for the specified use without further testing or modification. Customers are responsible for the design and operation of their applications and products using NXP Semiconductors products, and NXP Semiconductors accepts no liability for any assistance with applications or customer product design. It is customer's sole responsibility to determine whether the NXP Semiconductors product is suitable and fit for the customer's applications and products planned, as well as for the planned application and use of customer's third party customer(s). Customers should provide appropriate design and operating safeguards to minimize the risks associated with their applications and products. NXP Semiconductors does not accept any liability related to any default, damage, costs or problem which is based on any weakness or default in the customer's applications or products, or the application or use by customer's third party customer(s). Customer is responsible for doing all necessary testing for the customer's applications and products using NXP Semiconductors products in order to avoid a default of the applications and the products or of the application or use by customer's third party customer(s). NXP does not accept any liability in this respect.

Suitability for use in automotive applications — This NXP Semiconductors product has been qualified for use in automotive applications. Unless otherwise agreed in writing, the product is not designed, authorized or warranted to be suitable for use in life support, life-critical or safety-critical systems or equipment, nor in applications where failure or malfunction of an NXP Semiconductors product can reasonably be expected to result in personal injury, death or severe property or environmental damage. NXP Semiconductors and its suppliers accept no liability for inclusion and/or use of NXP Semiconductors products in such equipment or applications and therefore such inclusion and/or use is at the customer's own risk.

Export control — This document as well as the item(s) described herein may be subject to export control regulations. Export might require a prior authorization from competent authorities.

Translations — A non-English (translated) version of a document is for reference only. The English version shall prevail in case of any discrepancy between the translated and English versions.

Security — While NXP Semiconductors has implemented advanced security features, all products may be subject to unidentified vulnerabilities. Customers are responsible for the design and operation of their applications and products to reduce the effect of these vulnerabilities on customer's applications and products, and NXP Semiconductors accepts no liability for any vulnerability that is discovered. Customers should implement appropriate design and operating safeguards to minimize the risks associated with their applications and products.

7.3 Trademarks

Notice: All referenced brands, product names, service names and trademarks are the property of their respective owners.

NXP — is a trademark of NXP B.V.

All information provided in this document is subject to legal disclaimers.

© NXP B.V. 2020. All rights reserved.

Tables

Tab. 1. CRC polynomial and seed 2
Tab. 2. Command and response mode - CRC calculation examples 6
Tab. 3. Register read command message format 6
Tab. 4. Register read response message format 7
Tab. 5. Register write command message format 8
Tab. 6. Register write response message format 9
Tab. 7. Sensor data request command message format 10
Tab. 8. Sensor data request response message format 10
Tab. 9. Revision history 11

Figures

Fig. 1. Standard 32-bit SPI protocol timing diagram 1
Fig. 2. Register read command of 0x3E and 0x3F (WHO_AM_I and I2C_ADDRESS) 8
Fig. 3. Register write command of 0x24 in the register 0x42 9
Fig. 4. Data request command with a sensor source ID = 1 11

All information provided in this document is subject to legal disclaimers.

© NXP B.V. 2020. All rights reserved.

Contents

1 Introduction 1
2 Procedure for 8-bit CRC 2
3 Register read command 6
3.1 Register read command message format 6
3.2 Register read response message format 7
4 Register write command 8
4.1 Register write command message format 8
4.2 Register write response message format 9
5 Data request command (direct access to sensor data pressure) 9
5.1 Sensor data request command message format 10
5.2 Sensor data request response message format 10
6 Revision history 11
7 Legal information 12

Please be aware that important notices concerning this document and the product(s) described herein, have been included in section 'Legal information'.

© NXP B.V. 2020. All rights reserved.

For more information, please visit: http://www.nxp.com
For sales office addresses, please send an email to: salesaddresses@nxp.com

Date of release: 1 April 2020
Document identifier: AN12731